

```
#include <stdio.h>
#include <stdlib.h>

int disk[100];

int contiguous(int n, int size, int blocks[])
{
    int i, count = 0, start = -1;

    for (i = 0; i < n; i++)
    {
        if (disk[i] == 0)
            count++;
        else
            count = 0;

        if (count == size)
        {
            start = i - size + 1;
            break;
        }
    }

    if (start == -1)
        return 0;

    for (i = 0; i < size; i++)
    {
        blocks[i] = start + i;
        disk[start + i] = 1;
    }

    return 1;
}

int indexed(int n, int size, int blocks[])
{
    int i, count = 0;
    int indexBlock = -1;

    /* Find index block */
    for (i = 0; i < n; i++)
    {
        if (disk[i] == 0)
        {
            indexBlock = i;
            disk[i] = 1;
            break;
        }
    }
}
```

```
    }
}

if (indexBlock == -1)
    return 0;

/* Find data blocks */
for (i = 0; i < n && count < size; i++)
{
    if (disk[i] == 0)
    {
        blocks[count] = i;
        disk[i] = 1;
        count++;
    }
}

if (count < size)
    return 0;

return 1;
}

int fcfs(int blocks[], int n, int head)
{
    int i, current = head;
    int movement = 0;

    printf("FCFS Sequence: %d ", head);

    for (i = 0; i < n; i++)
    {
        movement += abs(current - blocks[i]);
        current = blocks[i];
        printf("-> %d ", current);
    }

    printf("\nTotal Seek Time = %d\n", movement);

    return movement;
}

int sstf(int blocks[], int n, int head)
{
    int visited[100] = {0};
    int i, j, min, index;
    int current = head;
    int movement = 0;
```

```
printf("SSTF Sequence: %d ", head);

for (i = 0; i < n; i++)
{
    min = 9999;
    index = -1;

    for (j = 0; j < n; j++)
    {
        if (!visited[j])
        {
            if (abs(current - blocks[j]) < min)
            {
                min = abs(current - blocks[j]);
                index = j;
            }
        }
    }

    visited[index] = 1;
    movement += min;
    current = blocks[index];

    printf("-> %d ", current);
}

printf("\nTotal Seek Time = %d\n", movement);

return movement;
}

int main()
{
    int n, fileSize;
    int head;
    int allocation, scheduling;
    int blocks[100];
    int i, result, seek;

    printf("Enter total number of disk blocks: ");
    scanf("%d", &n);

    printf("Enter file size: ");
    scanf("%d", &fileSize);

    printf("\nAllocation Method\n");
    printf("1. Contiguous\n");
    printf("2. Indexed\n");
    printf("Enter choice: ");
```

```
scanf("%d", &allocation);

printf("\nScheduling Method\n");
printf("1. FCFS\n");
printf("2. SSTF\n");
printf("Enter choice: ");
scanf("%d", &scheduling);

printf("\nEnter initial head position: ");
scanf("%d", &head);

/* Initialize disk */
for (i = 0; i < n; i++)
    disk[i] = 0;

/* Allocation */
if (allocation == 1)
{
    result = contiguous(n, fileSize, blocks);
}
else if (allocation == 2)
{
    result = indexed(n, fileSize, blocks);
}
else
{
    printf("Invalid allocation method.\n");
    return 0;
}

if (!result)
{
    printf("File allocation failed.\n");
    return 0;
}

printf("\nAllocated Blocks: ");

for (i = 0; i < fileSize; i++)
    printf("%d ", blocks[i]);

printf("\n\n");

/* Scheduling */
if (scheduling == 1)
{
    seek = fcfs(blocks, fileSize, head);
}
else if (scheduling == 2)
```

```
{
    seek = sstf(blocks, fileSize, head);
}
else
{
    printf("Invalid scheduling method.\n");
    return 0;
}

printf("\nFinal Total Seek Time = %d\n", seek);

return 0;
}
```